

Project: AWS Cost Monitoring & Budget Alerts

Services Used:

AWS Budgets, Amazon SNS, AWS Lambda, Amazon SES, Cost Explorer

Description:

This project implements an automated cost monitoring system in AWS that tracks cloud spending and sends real-time email alerts when defined budget thresholds are exceeded. It uses AWS Budgets to monitor usage, SNS to trigger events, Lambda to process alerts, and SES to send email notifications.

Definition:

The AWS Cost Monitoring & Budget Alerts system is designed to monitor AWS billing and notify users when spending crosses predefined thresholds. It integrates AWS Budgets with serverless services to provide automated, real-time alerts and improve cost control.

Scope of the Project:

- Automated cost tracking using AWS Budgets
- Threshold-based alert system (50%, 80%, 100%)
- Real-time notifications using SNS and SES
- Serverless processing using Lambda
- Optional cost insights using Cost Explorer

This Project Covers:

- **Budget Monitoring:** Tracks AWS usage against a defined budget
- **Alert Automation:** Sends alerts when thresholds are reached
- **Event-Driven Architecture:** Uses SNS and Lambda for automation
- **Email Notification System:** Sends alerts via SES
- **Cost Visibility:** Provides insights into cloud spending

Methodology:

1. AWS Budgets + SNS + Lambda + SES (Implemented Method)

- Uses AWS managed services
- Fully automated alerting system
- Serverless architecture
- Scalable and cost-effective
- Sends real-time notifications

2. Manual Monitoring + Email Alerts (Alternative Method)

- Uses manual checking of billing dashboard
- No automation

- Delayed response to cost increase
- Not efficient for production environments
- This method was not implemented

AWS Services Used in the Project:

1. AWS Budgets

Definition: AWS Budgets allows users to set custom cost and usage budgets and receive alerts when thresholds are exceeded.

Purpose: Monitor AWS spending

Role: Triggers alerts when budget limits are crossed

2. Amazon SNS (Simple Notification Service)

Definition: A messaging service used to send notifications via multiple protocols like email, SMS, and Lambda triggers.

Purpose: Sends alerts and triggers events

Role: Receives budget alerts and invokes Lambda

3. AWS Lambda

Definition: A serverless compute service that runs code without managing servers.

Purpose: Processes budget alerts

Role: Formats alert messages and sends emails via SES

4. Amazon SES (Simple Email Service)

Definition: A cloud-based email service for sending transactional emails.

Purpose: Send email notifications

Role: Delivers alert emails to users

5. AWS Cost Explorer

Definition: A tool to visualize and analyze AWS costs and usage.

Purpose: Provides cost insights

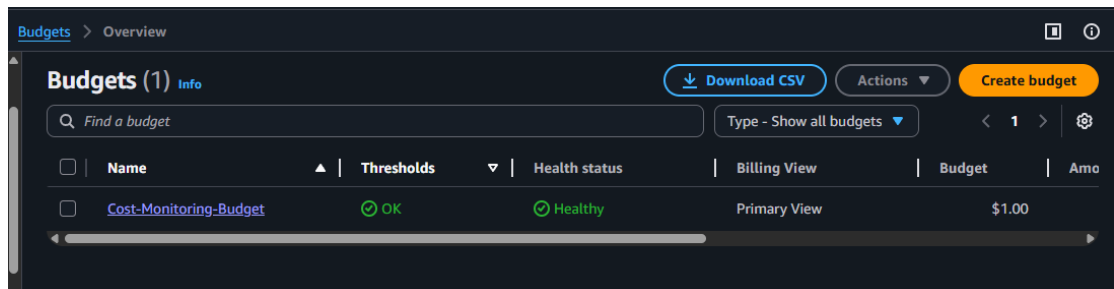
Role: (Optional) Used by Lambda to fetch detailed spending data



Implementation Steps:

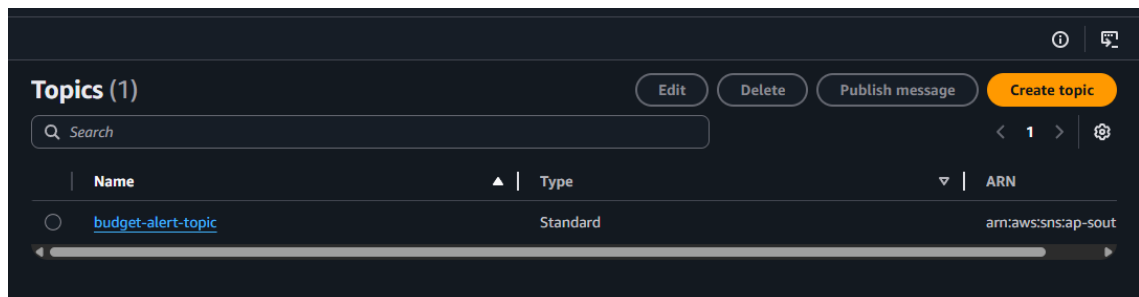
Step 1: Create Budget

- Go to AWS Budgets
- Select **Customize (Advanced)**
- Choose **Cost Budget**
- Set monthly budget (e.g., \$1 for testing)



Step 3: Create SNS Topic

- Go to SNS → Create Topic
- Name: budget-alert-topic
- Create email subscription
- Confirm subscription via email



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

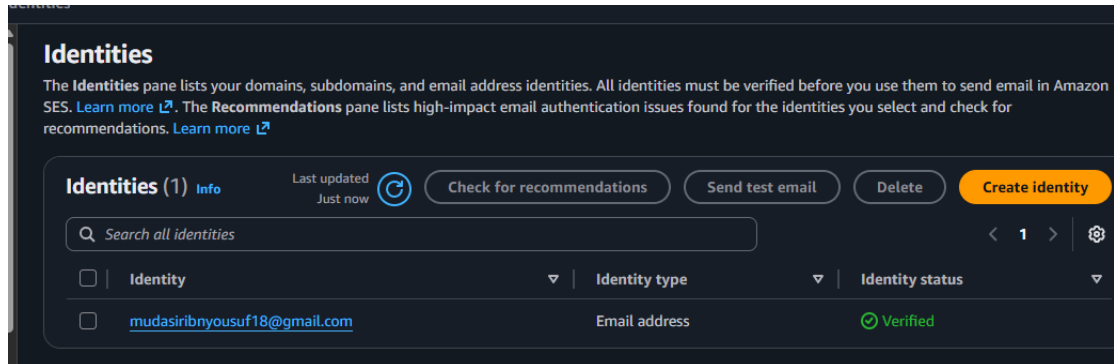
Your subscription's id is:

arn:aws:sns:ap-south-1:356302822046:budget-alert-topic:b3e18cb2-5236-4bda-9666-9713dd13bebb

If it was not your intention to subscribe, [click here to unsubscribe](#).

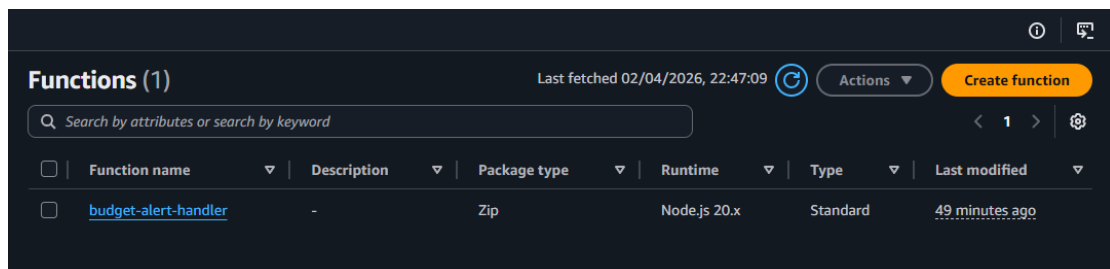
Step 4: Configure SES

- Go to SES → Verified identities
- Verify email address
- Confirm via email link
- Ensure region matches Lambda



Step 5: Create Lambda Function

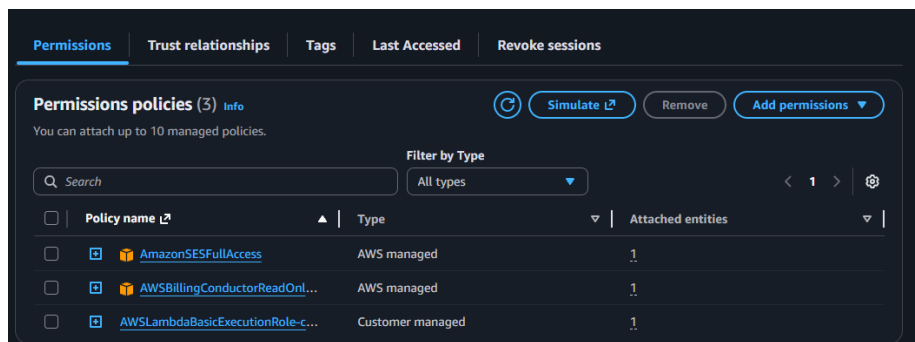
- Go to Lambda → Create Function
- Runtime: Node.js
- Function name: budget-alert-handler
- Assign execution role



Step 6: Configure Permissions

Attach policies:

- AmazonSESFullAccess
- AWSBillingReadOnlyAccess
- AWSLambdaBasicExecutionRole



Step 7: Add SNS Trigger

- Add SNS trigger to Lambda
- Select budget-alert-topic

Add trigger

Trigger configuration [Info](#)

 SNS
aws asynchronous messaging notifications pub-sub push

SNS topic

Select the SNS topic to subscribe to.

✕ ↻

Lambda will add the necessary permissions for AWS SNS to invoke your Lambda function from this trigger.

[Info](#)

Step 9: Deploy code and Test

- Deploy Lambda code
- Create SNS test event
- Run test
- Verify email received

```
JS index.mjs X
JS index.mjs > [0] handler > [0] params > Message
1 import { SESClient, SendEmailCommand } from "@aws-sdk/client-ses";
2
3 const ses = new SESClient({ region: "ap-south-1" });
4
5 export const handler = async (event) => {
6   try {
7     const snsMessage =
8       event?.Records?.[0]?.Sns?.Message || "Test alert: no SNS message found.";
9
10    const params = {
11      Source: "mudasiribnyousuf18@gmail.com",
12      Destination: {
13        ToAddresses: ["mudasiribnyousuf18@gmail.com"],
14      },
15      Message: {
16        Subject: {
17          Data: "AWS Budget Alert",
18        },
19        Body: {
20          Text: {
21            Data: `AWS Budget Alert Triggered\n\nMessage:\n${snsMessage}`,
22          },
23        },
24      },
25    };
26
27    await ses.send(new SendEmailCommand(params));
28
29    return {
30      statusCode: 200,
31      body: "Email sent successfully",
32    };
33   } catch (error) {
34     console.error("Error:", error);
35     throw error;
36   }
37 }
```

```

Status: Succeeded
Test Event Name: new

Response:
{
  "statusCode": 200,
  "body": "Email sent successfully"
}

The area below shows the last 4 KB of the execution log.

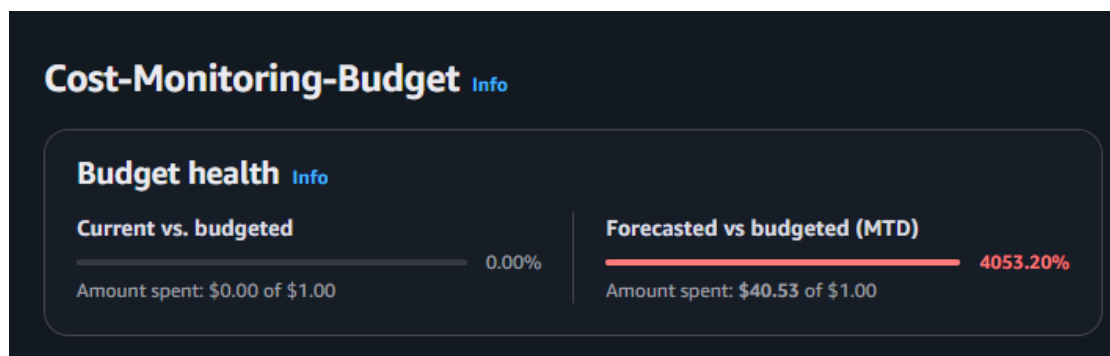
Function Logs:
START RequestId: f5c70445-fc94-4de3-81ac-ef2587e3edde Version: $LATEST
END RequestId: f5c70445-fc94-4de3-81ac-ef2587e3edde
REPORT RequestId: f5c70445-fc94-4de3-81ac-ef2587e3edde  Duration: 986.45 ms Billed Duration: 1419 ms    Memory Size: 128 MB Max Memory Use
Request ID: f5c70445-fc94-4de3-81ac-ef2587e3edde

```

Lambda Function Logic:

- Receive SNS event
- Extract budget alert message
- Format notification message
- Send email using SES
- Handle errors and log output

Cost Monitoring:



Left side:

- **Actual spend: \$0.00 / \$1.00**
You have **not used any cost yet**

Right side (IMPORTANT):

- **Forecasted spend: \$40.53 / \$1.00**
AWS is predicting you may spend **\$40 this month**

Why No Notification Was Triggered

The budget alert was not triggered because notifications are configured based on **actual spending**, not forecasted spending.

At the time of observation:

- **Actual Spend:** \$0.00
- **50% Threshold Value:** \$0.50

Since the actual spend (\$0.00) is below the defined threshold (\$0.50), the alert condition has not been met.

Troubleshooting:

Issue 1: Email not sending

Cause: Email not verified in SES

Solution: Verify email in SES and confirm via inbox

Issue 2: Region mismatch

Cause: SES and Lambda in different regions

Solution: Ensure both are in same region (e.g., ap-south-1)

Issue 3: Lambda test error (SNS format)

Cause: Incorrect test event format

Solution: Use SNS event structure

Issue 4: Permissions error

Cause: Missing IAM permissions

Solution: Attach required policies to Lambda role

Conclusion:

This project successfully demonstrates an automated AWS cost monitoring system that tracks usage and sends real-time alerts when budget thresholds are exceeded. By integrating AWS Budgets, SNS, Lambda, and SES, the system ensures efficient cost control, reduces manual monitoring, and provides a scalable serverless solution suitable for real-world applications.